

INFO0603

COMPRESSION ET CRYPTOGRAPHIE

COURS 6 FONCTIONS DE HACHAGE



Goéry Valance
Département de Mathématiques et Informatique
Février 2023

Les différents problèmes de sécurité informatique

Introduction à la sécurité Sécurité

Confidentialité, intégrité et authentification

Confidentialité

- Concerne toutes les parties du système : matériel, logiciels, données
- Accès uniquement aux parties autorisées
- Sécurisation des données

Intégrité

- Modifications suivant autorisation
- S'assurer de la non modification des données/messages

Authentification

- S'assurer de l'identité de l'acteur
- Associée à des autorisations

O. Flauzac / C. Rabat (Info0503)

Sécurisation des applications réparties

2020-2021

6 / 51



12/01/2023

Info0603 Compression et Cryptographie - Cours 6

2/22

Les différents problèmes de sécurité informatique

Introduction à la sécurité Sécurité

Ce que l'on veut éviter

- **Interception :**
 - Accès à des parties non autorisées
 - Récupération de données (écoute, copie de données)
 - **Interruption :**
 - Service inaccessible : déni de service (distribué)
 - Données corrompues
 - **Modification :** modification non autorisées
 - **Ajout :** ajout d'informations non autorisées (injection)
- Contrôler une identité
 - Chiffrer les données
 - Contrôler les données

O. Flauzac / C. Rabat (Info0503)

Sécurisation des applications réparties

2020-2021

7 / 51

12/01/2023

Info0603 Compression et Cryptographie - Cours 6

3/22

Les codes non injectifs

Les codes non injectifs

Nous n'avons, pour le moment, travaillé que sur des codes injectifs :

$$F(m)=F(m')\Rightarrow m=m'$$

Soit en contraposée : $m \neq m' \Rightarrow F(m) \neq F(m')$

Mais des codes NON injectifs peuvent être utiles
(cad qu'il existe (pas souvent) m et m' tels que $m \neq m'$
pour lesquels $F(m) = F(m')$)

Voir Théorie des Codes chez Dunod

12/01/2023

Info0603 Compression et Cryptographie - Cours 6

4/22

On parle alors d'**empreinte** d'un message, utile pour :

- Détection d'erreur
- Détection de modifications
- Signature
- Authentification

Codes de contrôle

L'exemple le plus trivial est le bit de parité...

Le numéro de carte bancaire lui, est un identifiant unique structuré selon la norme ISO/IEC 7812, qui se décompose ainsi :

- les six premiers chiffres constituent le numéro d'émetteur désigné par IIN pour Issuer Identification Number. Le premier chiffre identifie le type de carte : 3 pour les cartes American Express, 4 pour les cartes Visa, 5 pour les cartes Mastercard.
- Les chiffres suivants (9 à 12 chiffres) constituent l'identifiant de la carte chez l'émetteur et sont attribués par l'émetteur lui-même. Les banques françaises utilisent exactement 9 chiffres comme identifiant dans la très grande majorité des cas.
- Le dernier chiffre est **une clé de contrôle** permettant de vérifier que le numéro de carte est conforme à la norme. Le code complet est évalué selon la **formule de Luhn et doit donner 0**.

<https://www.comprendrelespaiements.com/la-carte-bancaire-la-connaissiez-vous/>

Mais la raison d'être de la clé de contrôle n'est pas de résister à la collision.



Fonctions de hachage

On parlera de fonction de hachage pour des empreintes plus évoluées, car destinées à **identifier** le message. C'est un résumé du message qui permettra, non de le reconstituer, mais de l'identifier si on dispose au préalable d'une table de correspondance (penser aux empreintes digitales).

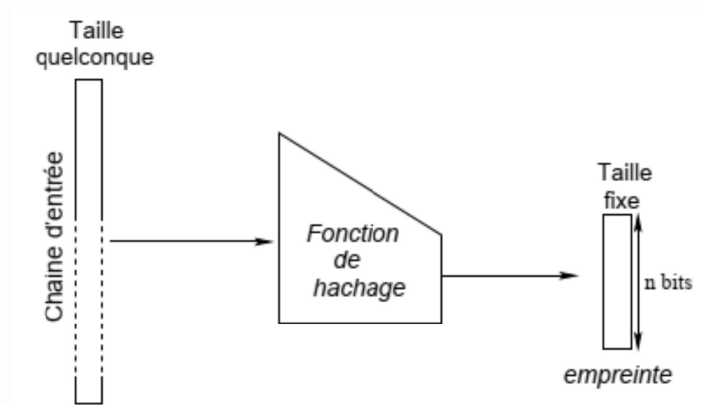
etc/shadow :

toto:

\$6\$loDTtoA6C5byWY7\$op5a67Z.HxuOxtQljDBvjN8aFi0muQ7KtXlamWtiyj3g8im5dEB6nIQDAQgUT5s
GZUqcTIqLf9L9vS4NcgUo

Fonctions de hachage

Formellement une fonction de hachage $H : \{0, 1\}^* \longrightarrow \{0, 1\}^n$
Est une application qui transforme une chaîne de taille
quelconque en une chaîne de taille fixe n :



Une fonction de Hachage n'est pas **Bijective**.

Une fonction de hachage peut-être **surjective** (c'est souhaitable) mais ne peut pas être **injective** (les doublons sont inévitables).

On parle de **collision** entre x et x' lorsque
 $x \neq x'$ et $H(x) = H(x')$

Les collisions sont inévitables (mais on n'en connaît aucune!).

En notant $y = H(x)$, y est l'**empreinte** de x ,
et x est un **antécédent** (ou une **pré-image**) de y .

Fonctions de hachage

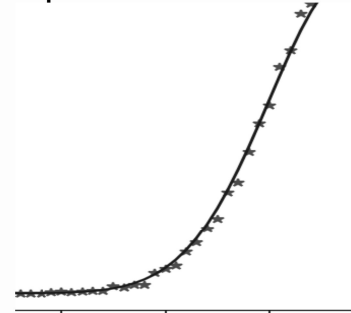
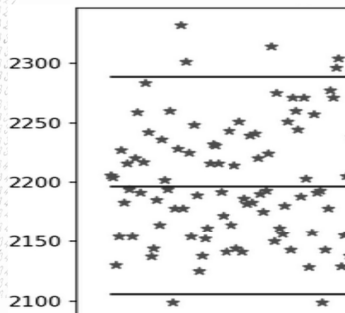
Pour être utile une fonction de hachage doit être facile à calculer mais il doit être difficile (par ordre croissant) de :

- Découvrir une **collision** (découvrir x et x' tel que $H(x)=H(x')$)
<<<
- Trouver une **pré-image** (y donné trouver x tel que $H(x)=y$)
<<<
- Trouver une **seconde pré-image** (x donné, trouver x' tel que $H(x)=H(x')$)

Fonctions de hachage

Pour cela, modifier un tant soit peu un message doit modifier considérablement la valeur de hachage et ce de manière pratiquement imprévisible.

C'est pour cela que certains générateurs de nombres aléatoires utilisent de « bonnes » fonctions de hachage cryptographique.



Les clés utilisées par les algorithmes de chiffrement modernes sont des chaînes aléatoires d'au moins 128 bits, difficiles à retenir pour un humain.

Pour pallier ce problème, elles peuvent être calculées en hachant un mot de passe. Ce mot de passe doit idéalement être suffisamment long pour contenir 128 bits d'entropie.

Recherche de doublons, paradoxe des anniversaires

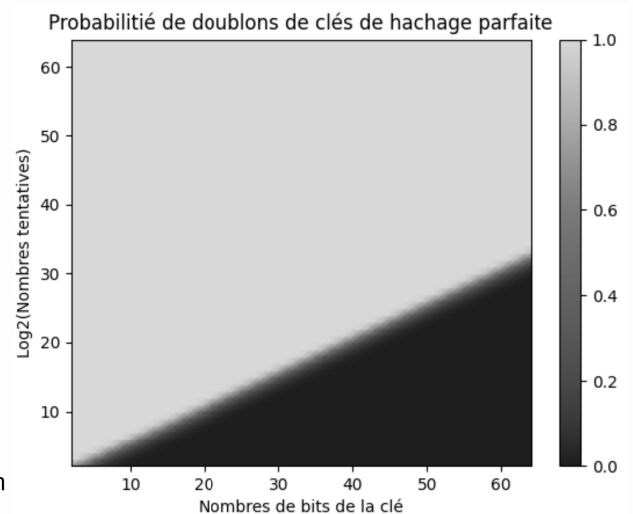
En notant p la probabilité d'avoir un doublon avec n valeurs tirées aléatoirement parmi N :

$$p(n) \approx 1 - e^{-\frac{n(n-1)}{2 \cdot N}} \quad \text{et} \quad n(p) \approx \sqrt{2 \cdot N \ln\left(\frac{1}{1-p}\right)}$$

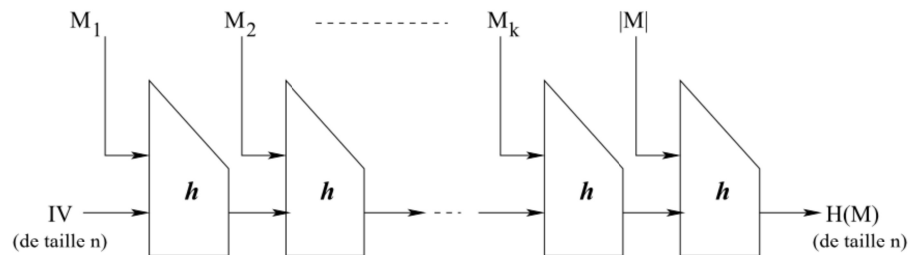
Avec p constant on a donc : $2 \cdot N \approx n(n-1)$

Un doublon n'arrivera donc avec SHA256 qu'après un nombre de calculs de l'ordre de 2^{128} .

Actuellement le Bitcoin mobilise 200EH/s $\approx 2^{68}$ H/s soit 2^{93} H/an



Soit h un hachage défini sur une taille fixe.
On peut l'étendre à H défini sur une taille quelconque par la construction de Merkle-Damgård car on conserve sa résistance à la collision :



Où IV (Initial value) est une chaîne de taille fixe fixée par l'implémentation.

Hachage par le logarithme discret (Chaum, von Heijst et Pfitzmann):

Soit la fonction h définie pour p premier et $q = \frac{p-1}{2}$ premier aussi,

α et β deux éléments primitifs de F_p (c'est à dire d'ordre p) par :

$$h : F_q \times F_p \rightarrow F_p^* \\ (x_1, x_2) \mapsto \alpha^{x_1} \beta^{x_2}$$

C'est une fonction résistante à la collision (tant que le calcul de $\log_\alpha$ est NP).

Hachage par chiffrement symétrique

Finalement, trouver une fonction de hachage performante est assez aisée car :

Tout chiffrement symétrique permet de définir une fonction de hachage !

Hachage par chiffrement symétrique

Hachage par chiffrement symétrique :

(Théorie des codes Dunod)

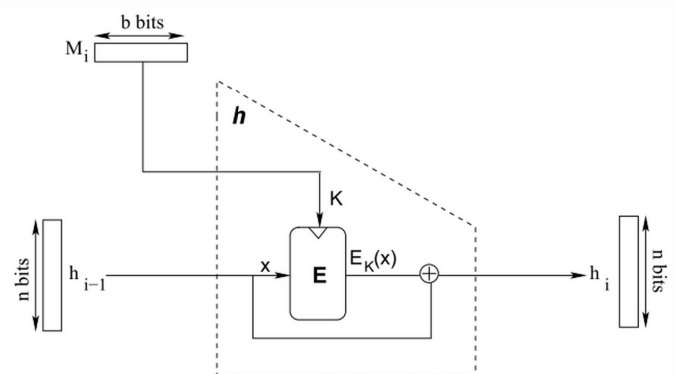


FIG. 1.14: Construction de Davies-Meyer.

Hachage par chiffrement symétrique

Hachage par chiffrement symétrique :

Une méthode plus robuste :

(moins vulnérable aux attaques sur les points fixes)

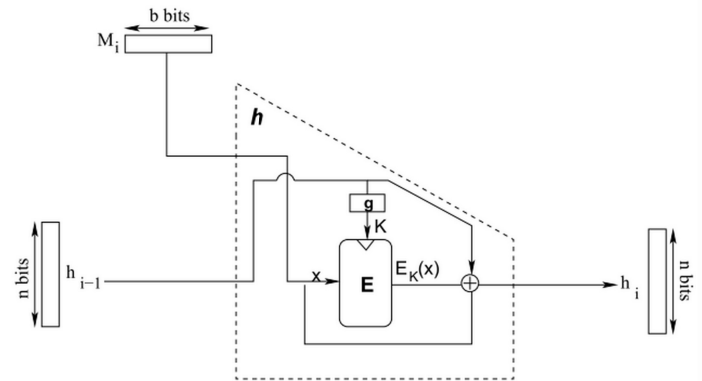


FIG. 1.15: Construction de Miyaguchi-Preneel.

(Théorie des codes Dunod)

Hachage MD5

19991 : La taille d'empreinte dans MD5 est de 128 bits. La recherche de collisions par force brute (attaque de Yuval) a donc une complexité de 2^{64} calculs d'empreintes.

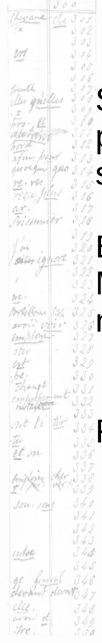
1996 : Une possibilité de créer des collisions à la demande est découverte.

2004 : une équipe chinoise découvre des collisions complètes.

Cependant, la fonction MD5 reste encore largement utilisée comme outil de vérification lors des téléchargements. MD5 a été employé dans GNU/Linux (avec la présence d'un sel) pour stocker les empreintes des mots de passe. La commande enable secret des commutateurs et routeurs Cisco, utilisait le hachage MD5 pour stocker le mot de passe du mode privilégié dans le fichier de configuration de l'équipement (wikipedia)

<https://www.frameip.com/decrypter-dechiffrer-cracker-hash-md5/>

Hachage SHA



SHA-1 (Secure Hash Algorithm) est une fonction de hachage de 160 bits conçue par la National Security Agency des États-Unis (NSA), et publiée comme un standard fédéral. Elle produit un résultat (appelé « hash » ou condensat) .

En 2005, des cryptanalystes ont découvert des attaques sur SHA-1. Microsoft, Google et Mozilla ont cessé d'accepter les certificats SHA-1 dans leurs navigateurs en 2017.

Regardons tout de même son algorithme...

BLOCK CHAIN